CIS Center for Internet Security®

CIS Benchmarks™

CIS Amazon Elastic Kubernetes Service (EKS) Benchmark

v1.2.0 - 11-23-2022

Terms of Use

Please see the below link for our current terms of use:

https://www.cisecurity.org/cis-securesuite/cis-securesuite-membership-terms-of-use/

Table of Contents

Overview

All CIS Benchmarks focus on technical configuration settings used to maintain and/or increase the security of the addressed technology, and they should be used in **conjunction** with other essential cyber hygiene tasks like:

- Monitoring the base operating system for vulnerabilities and quickly updating with the latest security patches
- Monitoring applications and libraries for vulnerabilities and quickly updating with the latest security patches

In the end, the CIS Benchmarks are designed as a key **component** of a comprehensive cybersecurity program.

This document provides prescriptive guidance for running Amazon Elastic Kubernetes Service (EKS) following recommended security controls. This benchmark only includes controls which can be modified by an end user of Amazon EKS.
To obtain the latest version of this guide, please visit www.cisecurity.org. If you have questions, comments, or have identified ways to improve this guide, please write us at support@cisecurity.org.

Intended Audience

This document is intended for cluster administrators, security specialists, auditors, and any personnel who plan to develop, deploy, assess, or secure solutions that incorporate Amazon EKS using managed and self-managed nodes.
Customers using Amazon EKS on AWS Fargate are not responsible for node management. Hence, this document is not scoped for Amazon EKS on AWS Fargate customers.

Consensus Guidance

This CIS Benchmark was created using a consensus review process comprised of a global community of subject matter experts. The process combines real world experience with data-based information to create technology specific guidance to assist users to secure their environments. Consensus participants provide perspective from a diverse set of backgrounds including consulting, software development, audit and compliance, security research, operations, government, and legal.

Each CIS Benchmark undergoes two phases of consensus review. The first phase occurs during initial Benchmark development. During this phase, subject matter experts convene to discuss, create, and test working drafts of the Benchmark. This discussion occurs until consensus has been reached on Benchmark recommendations. The second phase begins after the Benchmark has been published. During this phase, all feedback provided by the Internet community is reviewed by the consensus team for incorporation in the Benchmark. If you are interested in participating in the consensus process, please visit https://workbench.cisecurity.org/.

Typographical Conventions

The following typographical conventions are used throughout this guide:

Convention	Meaning
`Stylized Monospace font`	Used for blocks of code, command, and script examples. Text should be interpreted exactly as presented.
`Monospace font`	Used for inline code, commands, or examples. Text should be interpreted exactly as presented.
<italic font in brackets>	Italic texts set in angle brackets denote a variable requiring substitution for a real value.
Italic font	Used to denote the title of a book, article, or other publication.
Note	Additional information or caveats

Recommendation Definitions

The following defines the various components included in a CIS recommendation as applicable. If any of the components are not applicable it will be noted or the component will not be included in the recommendation.

Title

Concise description for the recommendation's intended configuration.

Assessment Status

An assessment status is included for every recommendation. The assessment status indicates whether the given recommendation can be automated or requires manual steps to implement. Both statuses are equally important and are determined and supported as defined below:

Automated

Represents recommendations for which assessment of a technical control can be fully automated and validated to a pass/fail state. Recommendations will include the necessary information to implement automation.

Manual

Represents recommendations for which assessment of a technical control cannot be fully automated and requires all or some manual steps to validate that the configured state is set as expected. The expected state can vary depending on the environment.

Profile

A collection of recommendations for securing a technology or a supporting platform. Most benchmarks include at least a Level 1 and Level 2 Profile. Level 2 extends Level 1 recommendations and is not a standalone profile. The Profile Definitions section in the benchmark provides the definitions as they pertain to the recommendations included for the technology.

Description

Detailed information pertaining to the setting with which the recommendation is concerned. In some cases, the description will include the recommended value.

Rationale Statement

Detailed reasoning for the recommendation to provide the user a clear and concise understanding on the importance of the recommendation.

Impact Statement

Any security, functionality, or operational consequences that can result from following the recommendation.

Audit Procedure

Systematic instructions for determining if the target system complies with the recommendation

Remediation Procedure

Systematic instructions for applying recommendations to the target system to bring it into compliance according to the recommendation.

Default Value

Default value for the given setting in this recommendation, if known. If not known, either not configured or not defined will be applied.

References

Additional documentation relative to the recommendation.

CIS Critical Security Controls® (CIS Controls®)

The mapping between a recommendation and the CIS Controls is organized by CIS Controls version, Safeguard, and Implementation Group (IG). The Benchmark in its entirety addresses the CIS Controls safeguards of (v7) “5.1 - Establish Secure Configurations” and (v8) '4.1 - Establish and Maintain a Secure Configuration Process” so individual recommendations will not be mapped to these safeguards.

Additional Information

Supplementary information that does not correspond to any other field but may be useful to the user.

Profile Definitions

The following configuration profiles are defined by this Benchmark:

- **Level 1**

  Level 1 profile for running Automated Assessment

- **Level 2**

  Level 2 profile for running Automated Assessment

Acknowledgements

This Benchmark exemplifies the great things a community of users, vendors, and subject matter experts can accomplish through consensus collaboration. The CIS community thanks the entire consensus team with special recognition to the following individuals who contributed greatly to the creation of this guide:

Thanks to Authors: Paavan Mistry and Randall Mowen
with special thanks to contributors: James Stocks, Daniel Burns, Rory McCune and Joe Bowbeer

Contributors

James Stocks
Daniel Burns
Mark Larinde
Joe Bowbeer
Hadas Yadayi
Rory McCune
Maya Gab
Nick Gibbon

Recommendations

1 Control Plane Components

Security is a shared responsibility between AWS and the Amazon EKS customer. The shared responsibility model describes this as security of the cloud and security in the cloud:

Security of the cloud – AWS is responsible for protecting the infrastructure that runs AWS services in the AWS Cloud. For Amazon EKS, AWS is responsible for the Kubernetes control plane, which includes the control plane nodes and etcd database. Third-party auditors regularly test and verify the effectiveness of our security as part of the AWS compliance programs. To learn about the compliance programs that apply to Amazon EKS, see AWS Services in Scope by Compliance Program.

Security in the cloud – Your responsibility includes the following areas.

- The security configuration of the data plane, including the configuration of the security groups that allow traffic to pass from the Amazon EKS control plane into the customer VPC
- The configuration of the worker nodes and the containers themselves
- The worker node guest operating system (including updates and security patches)
    - Amazon EKS follows the shared responsibility model for CVEs and security patches on managed node groups. Because managed nodes run the Amazon EKS-optimized AMIs, Amazon EKS is responsible for building patched versions of these AMIs when bugs or issues are reported and we are able to publish a fix. However, customers are responsible for deploying these patched AMI versions to your managed node groups.
- Other associated application software:
    - Setting up and managing network controls, such as firewall rules
    - Managing platform-level identity and access management, either with or in addition to IAM
- The sensitivity of your data, your company's requirements, and applicable laws and regulations

AWS is responsible for securing the control plane, though you might be able to configure certain options based on your requirements. Section 2 of this Benchmark addresses these configurations.

2 Control Plane Configuration

This section contains recommendations for Amazon EKS control plane logging configuration. Customers are able to configure logging for control plane in Amazon EKS.

2.1 Logging

2.1.1 Enable audit Logs (Manual)

Profile Applicability:

- Level 1

Description:

Control plane logs provide visibility into operation of the EKS Control plane component systems. The API server audit logs record all accepted and rejected requests in the cluster. When enabled via EKS configuration the control plane logs for a cluster are exported to a CloudWatch Log Group for persistence.

Rationale:

Audit logs enable visibility into all API server requests from authentic and anonymous sources. Stored log data can be analyzed manually or with tools to identify and understand anomalous or negative activity and lead to intelligent remediations.

Audit:

From Console:

1. For each EKS Cluster in each region;
2. Go to 'Amazon EKS' > 'Clusters' > 'CLUSTER_NAME' > 'Configuration' > 'Logging'.
3. This will show the control plane logging configuration:

```
API server: Enabled / Disabled
Audit: Enabled / Disabled
Authenticator: Enabled / Disabled
Controller manager: Enabled / Disabled
Scheduler: Enabled / Disabled
```

4. Ensure that all options are set to 'Enabled'.

From CLI:

```
# For each EKS Cluster in each region;
aws eks describe-cluster --name '${CLUSTER_NAME}' --query
'cluster.logging.clusterLogging' --region '${REGION_CODE}'
```

Remediation:

From Console:

1. For each EKS Cluster in each region;
2. Go to 'Amazon EKS' > 'Clusters' > '' > 'Configuration' > 'Logging'.
3. Click 'Manage logging'.

4. Ensure that all options are toggled to 'Enabled'.

```
API server: Enabled
Audit: Enabled
Authenticator: Enabled
Controller manager: Enabled
Scheduler: Enabled
```

5. Click 'Save Changes'.

From CLI:

```
# For each EKS Cluster in each region;
aws eks update-cluster-config \
    --region '${REGION_CODE}' \
    --name '${CLUSTER_NAME}' \
    --logging
'{"clusterLogging":[{"types":["api","audit","authenticator","controllerManage
r","scheduler"],"enabled":true}]}'
```

Default Value:

Control Plane Logging is disabled by default.

```
API server: Disabled
Audit: Disabled
Authenticator: Disabled
Controller manager: Disabled
Scheduler: Disabled
```

References:

1. https://kubernetes.io/docs/tasks/debug-application-cluster/audit/
2. https://aws.github.io/aws-eks-best-practices/detective/
3. https://docs.aws.amazon.com/eks/latest/userguide/control-plane-logs.html
4. https://docs.aws.amazon.com/eks/latest/userguide/logging-using-cloudtrail.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	8.1 Establish and Maintain an Audit Log Management Process Establish and maintain an audit log management process that defines the enterprise's logging requirements. At a minimum, address the collection, review, and retention of audit logs for enterprise assets. Review and update documentation annually, or when significant enterprise changes occur that could impact this Safeguard.	●	●	●

Controls Version	Control	IG 1	IG 2	IG 3
v8	8.2 Collect Audit Logs Collect audit logs. Ensure that logging, per the enterprise's audit log management process, has been enabled across enterprise assets.	●	●	●
v7	6.2 Activate audit logging Ensure that local logging has been enabled on all systems and networking devices.	●	●	●
v7	6.3 Enable Detailed Logging Enable system logging to include detailed information such as an event source, date, user, timestamp, source addresses, destination addresses, and other useful elements.		●	●

3 Worker Nodes

This section consists of security recommendations for the components that run on Amazon EKS worker nodes.

3.1 Worker Node Configuration Files

This section covers recommendations for configuration files on Amazon EKS worker nodes.

3.1.1 Ensure that the kubeconfig file permissions are set to 644 or more restrictive (Manual)

Profile Applicability:

- Level 1

Description:

If kubelet is running, and if it is configured by a kubeconfig file, ensure that the proxy kubeconfig file has permissions of 644 or more restrictive.

Rationale:

The `kubelet` kubeconfig file controls various parameters of the `kubelet` service in the worker node. You should restrict its file permissions to maintain the integrity of the file. The file should be writable by only the administrators on the system.

It is possible to run `kubelet` with the kubeconfig parameters configured as a Kubernetes ConfigMap instead of a file. In this case, there is no proxy kubeconfig file.

Impact:

None.

Audit:

SSH to the worker nodes
To check to see if the Kubelet Service is running:

```
sudo systemctl status kubelet
```

The output should return `Active: active (running) since..`
Run the following command on each node to find the appropriate kubeconfig file:

```
ps -ef | grep kubelet
```

The output of the above command should return something similar to `--kubeconfig /var/lib/kubelet/kubeconfig` which is the location of the kubeconfig file.
Run this command to obtain the kubeconfig file permissions:

```
stat -c %a /var/lib/kubelet/kubeconfig
```

The output of the above command gives you the kubeconfig file's permissions.
Verify that if a file is specified and it exists, the permissions are `644` or more restrictive.

Remediation:

Run the below command (based on the file location on your system) on the each worker node. For example,

```
chmod 644 <kubeconfig file>
```

Default Value:

See the AWS EKS documentation for the default value.

References:

1. https://kubernetes.io/docs/admin/kube-proxy/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 Configure Data Access Control Lists Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.	●	●	●
v7	5.2 Maintain Secure Images Maintain secure images or templates for all systems in the enterprise based on the organization's approved configuration standards. Any new system deployment or existing system that becomes compromised should be imaged using one of those images or templates.		●	●

3.1.2 Ensure that the kubelet kubeconfig file ownership is set to root:root (Manual)

Profile Applicability:

- Level 1

Description:

If `kubelet` is running, ensure that the file ownership of its kubeconfig file is set to `root:root`.

Rationale:

The kubeconfig file for `kubelet` controls various parameters for the `kubelet` service in the worker node. You should set its file ownership to maintain the integrity of the file. The file should be owned by `root:root`.

Impact:

None

Audit:

SSH to the worker nodes
To check to see if the Kubelet Service is running:

```
sudo systemctl status kubelet
```

The output should return `Active: active (running) since..`
Run the following command on each node to find the appropriate kubeconfig file:

```
ps -ef | grep kubelet
```

The output of the above command should return something similar to `--kubeconfig /var/lib/kubelet/kubeconfig` which is the location of the kubeconfig file.
Run this command to obtain the kubeconfig file ownership:

```
stat -c %U:%G /var/lib/kubelet/kubeconfig
```

The output of the above command gives you the kubeconfig file's ownership. Verify that the ownership is set to `root:root`.

Remediation:

Run the below command (based on the file location on your system) on each worker node.
For example,

```
chown root:root <proxy kubeconfig file>
```

Default Value:

See the AWS EKS documentation for the default value.

References:

1. https://kubernetes.io/docs/admin/kube-proxy/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 Configure Data Access Control Lists Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.	●	●	●
v7	5.2 Maintain Secure Images Maintain secure images or templates for all systems in the enterprise based on the organization's approved configuration standards. Any new system deployment or existing system that becomes compromised should be imaged using one of those images or templates.		●	●

3.1.3 Ensure that the kubelet configuration file has permissions set to 644 or more restrictive (Manual)

Profile Applicability:

- Level 1

Description:

Ensure that if the kubelet refers to a configuration file with the `--config` argument, that file has permissions of 644 or more restrictive.

Rationale:

The kubelet reads various parameters, including security settings, from a config file specified by the `--config` argument. If this file is specified you should restrict its file permissions to maintain the integrity of the file. The file should be writable by only the administrators on the system.

Impact:

None.

Audit:

First, SSH to the relevant worker node:
To check to see if the Kubelet Service is running:

```
sudo systemctl status kubelet
```

The output should return `Active: active (running) since..`
Run the following command on each node to find the appropriate Kubelet config file:

```
ps -ef | grep kubelet
```

The output of the above command should return something similar to `--config /etc/kubernetes/kubelet/kubelet-config.json` which is the location of the Kubelet config file.
Run the following command:

```
stat -c %a /etc/kubernetes/kubelet/kubelet-config.json
```

The output of the above command is the Kubelet config file's permissions. Verify that the permissions are `644` or more restrictive.

Remediation:

Run the following command (using the config file location identified in the Audit step)

```
chmod 644 /etc/kubernetes/kubelet/kubelet-config.json
```

Default Value:

See the AWS EKS documentation for the default value.

References:

1. https://kubernetes.io/docs/tasks/administer-cluster/kubelet-config-file/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 Configure Data Access Control Lists Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.	●	●	●
v7	5.2 Maintain Secure Images Maintain secure images or templates for all systems in the enterprise based on the organization's approved configuration standards. Any new system deployment or existing system that becomes compromised should be imaged using one of those images or templates.		●	●

3.1.4 Ensure that the kubelet configuration file ownership is set to root:root (Manual)

Profile Applicability:

- Level 1

Description:

Ensure that if the kubelet refers to a configuration file with the `--config` argument, that file is owned by root:root.

Rationale:

The kubelet reads various parameters, including security settings, from a config file specified by the `--config` argument. If this file is specified you should restrict its file permissions to maintain the integrity of the file. The file should be writable by only the administrators on the system.

Impact:

None

Audit:

First, SSH to the relevant worker node:
To check to see if the Kubelet Service is running:

```
sudo systemctl status kubelet
```

The output should return `Active: active (running) since..`
Run the following command on each node to find the appropriate Kubelet config file:

```
ps -ef | grep kubelet
```

The output of the above command should return something similar to `--config /etc/kubernetes/kubelet/kubelet-config.json` which is the location of the Kubelet config file.
Run the following command:

```
stat -c %U:%G /etc/kubernetes/kubelet/kubelet-config.json
```

The output of the above command is the Kubelet config file's ownership. Verify that the ownership is set to `root:root`

Remediation:

Run the following command (using the config file location identified in the Audit step)

```
chown root:root /etc/kubernetes/kubelet/kubelet-config.json
```

Default Value:

See the AWS EKS documentation for the default value.

References:

1. https://kubernetes.io/docs/admin/kube-proxy/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.3 Configure Data Access Control Lists Configure data access control lists based on a user's need to know. Apply data access control lists, also known as access permissions, to local and remote file systems, databases, and applications.	●	●	●
v7	5.2 Maintain Secure Images Maintain secure images or templates for all systems in the enterprise based on the organization's approved configuration standards. Any new system deployment or existing system that becomes compromised should be imaged using one of those images or templates.		●	●

3.2 Kubelet

Kubelets can accept configuration via a configuration file and in some cases via command line arguments. It is important to note that parameters provided as command line arguments will override their counterpart parameters in the configuration file (see `--config` details in the Kubelet CLI Reference for more info, where you can also find out which configuration parameters can be supplied as a command line argument).

With this in mind, it is important to check for the existence of command line arguments as well as configuration file entries when auditing Kubelet configuration.

Firstly, SSH to each node and execute the following command to find the Kubelet process:

```
ps -ef | grep kubelet
```

The output of the above command provides details of the active Kubelet process, from which we can see the command line arguments provided to the process. Also note the location of the configuration file, provided with the `--config` argument, as this will be needed to verify configuration. The file can be viewed with a command such as `more` or `less`, like so:

```
sudo less /path/to/kubelet-config.json
```

This config file could be in JSON or YAML format depending on your distribution.

3.2.1 Ensure that the Anonymous Auth is Not Enabled (Automated)

Profile Applicability:

- Level 1

Description:

Disable anonymous requests to the Kubelet server.

Rationale:

When enabled, requests that are not rejected by other configured authentication methods are treated as anonymous requests. These requests are then served by the Kubelet server. You should rely on authentication to authorize access and disallow anonymous requests.

Impact:

Anonymous requests will be rejected.

Audit:

Audit Method 1:
Kubelets can accept configuration via a configuration file and in some cases via command line arguments. It is important to note that parameters provided as command line arguments will override their counterpart parameters in the configuration file (see `--config` details in the Kubelet CLI Reference for more info, where you can also find out which configuration parameters can be supplied as a command line argument).
With this in mind, it is important to check for the existence of command line arguments as well as configuration file entries when auditing Kubelet configuration.
Firstly, SSH to each node and execute the following command to find the Kubelet process:

```
ps -ef | grep kubelet
```

The output of the above command provides details of the active Kubelet process, from which we can see the command line arguments provided to the process. Also note the location of the configuration file, provided with the `--config` argument, as this will be needed to verify configuration. The file can be viewed with a command such as `more` or `less`, like so:

```
sudo less /path/to/kubelet-config.json
```

Verify that Anonymous Authentication is not enabled. This may be configured as a command line argument to the kubelet service with `--anonymous-auth=false` or in the kubelet configuration file via `"authentication": { "anonymous": { "enabled": false }`.

Audit Method 2:

It is also possible to review the running configuration of a Kubelet via the /configz endpoint of the Kubernetes API. This can be achieved using `kubectl` to proxy your requests to the API.

Discover all nodes in your cluster by running the following command:

```
kubectl get nodes
```

Next, initiate a proxy with `kubectl` on a local port of your choice. In this example we will use 8080:

```
kubectl proxy --port=8080
```

With this running, in a separate terminal run the following command for each node:

```
export NODE_NAME=my-node-name
curl http://localhost:8080/api/v1/nodes/${NODE_NAME}/proxy/configz
```

The curl command will return the API response which will be a JSON formatted string representing the Kubelet configuration.

Verify that Anonymous Authentication is not enabled checking that `"authentication": { "anonymous": { "enabled": false }` is in the API response.

Remediation:

Remediation Method 1:

If configuring via the Kubelet config file, you first need to locate the file.

To do this, SSH to each node and execute the following command to find the kubelet process:

```
ps -ef | grep kubelet
```

The output of the above command provides details of the active kubelet process, from which we can see the location of the configuration file provided to the kubelet service with the `--config` argument. The file can be viewed with a command such as `more` or `less`, like so:

```
sudo less /path/to/kubelet-config.json
```

Disable Anonymous Authentication by setting the following parameter:

```
"authentication": { "anonymous": { "enabled": false } }
```

Remediation Method 2:
If using executable arguments, edit the kubelet service file on each worker node and ensure the below parameters are part of the `KUBELET_ARGS` variable string.
For systems using `systemd`, such as the Amazon EKS Optimised Amazon Linux or Bottlerocket AMIs, then this file can be found at `/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf`. Otherwise, you may need to look up documentation for your chosen operating system to determine which service manager is configured:

```
--anonymous-auth=false
```

For Both Remediation Steps:
Based on your system, restart the `kubelet` service and check the service status.
The following example is for operating systems using `systemd`, such as the Amazon EKS Optimised Amazon Linux or Bottlerocket AMIs, and invokes the `systemctl` command. If `systemctl` is not available then you will need to look up documentation for your chosen operating system to determine which service manager is configured:

```
systemctl daemon-reload
systemctl restart kubelet.service
systemctl status kubelet -l
```

Default Value:

See the EKS documentation for the default value.

References:

1. https://kubernetes.io/docs/reference/command-line-tools-reference/kubelet/
2. https://kubernetes.io/docs/reference/access-authn-authz/kubelet-authn-authz/#kubelet-authentication
3. https://kubernetes.io/docs/reference/config-api/kubelet-config.v1beta1/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.3 Disable Dormant Accounts Delete or disable any dormant accounts after a period of 45 days of inactivity, where supported.	●	●	●
v7	14.6 Protect Information through Access Control Lists Protect all information stored on systems with file system, network share, claims, application, or database specific access control lists. These controls will enforce the principle that only authorized individuals should have access to the information based on their need to access the information as a part of their responsibilities.	●	●	●

3.2.2 Ensure that the --authorization-mode argument is not set to AlwaysAllow (Automated)

Profile Applicability:

- Level 1

Description:

Do not allow all requests. Enable explicit authorization.

Rationale:

Kubelets can be configured to allow all authenticated requests (even anonymous ones) without needing explicit authorization checks from the apiserver. You should restrict this behavior and only allow explicitly authorized requests.

Impact:

Unauthorized requests will be denied.

Audit:

Audit Method 1:
Kubelets can accept configuration via a configuration file and in some cases via command line arguments. It is important to note that parameters provided as command line arguments will override their counterpart parameters in the configuration file (see `--config` details in the Kubelet CLI Reference for more info, where you can also find out which configuration parameters can be supplied as a command line argument).
With this in mind, it is important to check for the existence of command line arguments as well as configuration file entries when auditing Kubelet configuration.
Firstly, SSH to each node and execute the following command to find the Kubelet process:

```
ps -ef | grep kubelet
```

The output of the above command provides details of the active Kubelet process, from which we can see the command line arguments provided to the process. Also note the location of the configuration file, provided with the `--config` argument, as this will be needed to verify configuration. The file can be viewed with a command such as `more` or `less`, like so:

```
sudo less /path/to/kubelet-config.json
```

Verify that Webhook Authentication is enabled. This may be enabled as a command line argument to the kubelet service with `--authentication-token-webhook` or in the kubelet configuration file via `"authentication": { "webhook": { "enabled": true } }`.
Verify that the Authorization Mode is set to `WebHook`. This may be set as a command line argument to the kubelet service with `--authorization-mode=Webhook` or in the configuration file via `"authorization": { "mode": "Webhook }`.

Audit Method 2:

It is also possible to review the running configuration of a Kubelet via the /configz endpoint of the Kubernetes API. This can be achieved using `kubectl` to proxy your requests to the API.
Discover all nodes in your cluster by running the following command:

```
kubectl get nodes
```

Next, initiate a proxy with kubectl on a local port of your choice. In this example we will use 8080:

```
kubectl proxy --port=8080
```

With this running, in a separate terminal run the following command for each node:

```
export NODE_NAME=my-node-name
curl http://localhost:8080/api/v1/nodes/${NODE_NAME}/proxy/configz
```

The curl command will return the API response which will be a JSON formatted string representing the Kubelet configuration.
Verify that Webhook Authentication is enabled with `"authentication": { "webhook": { "enabled": true } }` in the API response.
Verify that the Authorization Mode is set to `WebHook` with `"authorization": { "mode": "Webhook }` in the API response.

Remediation:

Remediation Method 1:

If configuring via the Kubelet config file, you first need to locate the file.
To do this, SSH to each node and execute the following command to find the kubelet process:

```
ps -ef | grep kubelet
```

The output of the above command provides details of the active kubelet process, from which we can see the location of the configuration file provided to the kubelet service with the `--config` argument. The file can be viewed with a command such as `more` or `less`, like so:

```
sudo less /path/to/kubelet-config.json
```

Enable Webhook Authentication by setting the following parameter:

```
"authentication": { "webhook": { "enabled": true } }
```

Next, set the Authorization Mode to `Webhook` by setting the following parameter:

```
"authorization": { "mode": "Webhook }
```

Finer detail of the `authentication` and `authorization` fields can be found in the Kubelet Configuration documentation.

Remediation Method 2:

If using executable arguments, edit the kubelet service file on each worker node and ensure the below parameters are part of the `KUBELET_ARGS` variable string.

For systems using `systemd`, such as the Amazon EKS Optimised Amazon Linux or Bottlerocket AMIs, then this file can be found at `/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf`. Otherwise, you may need to look up documentation for your chosen operating system to determine which service manager is configured:

```
--authentication-token-webhook
--authorization-mode=Webhook
```

For Both Remediation Steps:

Based on your system, restart the `kubelet` service and check the service status.

The following example is for operating systems using `systemd`, such as the Amazon EKS Optimised Amazon Linux or Bottlerocket AMIs, and invokes the `systemctl` command. If `systemctl` is not available then you will need to look up documentation for your chosen operating system to determine which service manager is configured:

```
systemctl daemon-reload
systemctl restart kubelet.service
systemctl status kubelet -l
```

Default Value:

See the EKS documentation for the default value.

References:

1. https://kubernetes.io/docs/reference/command-line-tools-reference/kubelet/
2. https://kubernetes.io/docs/reference/access-authn-authz/kubelet-authn-authz/#kubelet-authentication
3. https://kubernetes.io/docs/reference/config-api/kubelet-config.v1beta1/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	5.4 Restrict Administrator Privileges to Dedicated Administrator Accounts Restrict administrator privileges to dedicated administrator accounts on enterprise assets. Conduct general computing activities, such as internet browsing, email, and productivity suite use, from the user's primary, non-privileged account.	●	●	●

Controls Version	Control	IG 1	IG 2	IG 3
v7	4.2 Change Default Passwords Before deploying any new asset, change all default passwords to have values consistent with administrative level accounts.	●	●	●

3.2.3 Ensure that a Client CA File is Configured (Manual)

Profile Applicability:

- Level 1

Description:

Enable Kubelet authentication using certificates.

Rationale:

The connections from the apiserver to the kubelet are used for fetching logs for pods, attaching (through kubectl) to running pods, and using the kubelet's port-forwarding functionality. These connections terminate at the kubelet's HTTPS endpoint. By default, the apiserver does not verify the kubelet's serving certificate, which makes the connection subject to man-in-the-middle attacks, and unsafe to run over untrusted and/or public networks. Enabling Kubelet certificate authentication ensures that the apiserver could authenticate the Kubelet before submitting any requests.

Impact:

You require TLS to be configured on apiserver as well as kubelets.

Audit:

Audit Method 1:
Kubelets can accept configuration via a configuration file and in some cases via command line arguments. It is important to note that parameters provided as command line arguments will override their counterpart parameters in the configuration file (see `--config` details in the Kubelet CLI Reference for more info, where you can also find out which configuration parameters can be supplied as a command line argument).
With this in mind, it is important to check for the existence of command line arguments as well as configuration file entries when auditing Kubelet configuration.
Firstly, SSH to each node and execute the following command to find the Kubelet process:

```
ps -ef | grep kubelet
```

The output of the above command provides details of the active Kubelet process, from which we can see the command line arguments provided to the process. Also note the location of the configuration file, provided with the `--config` argument, as this will be needed to verify configuration. The file can be viewed with a command such as `more` or `less`, like so:

```
sudo less /path/to/kubelet-config.json
```

Verify that a client certificate authority file is configured. This may be configured using a command line argument to the kubelet service with `--client-ca-file` or in the kubelet configuration file via `"authentication": { "x509": {"clientCAFile": <path/to/client-ca-file> } }`.

Audit Method 2:

It is also possible to review the running configuration of a Kubelet via the /configz endpoint of the Kubernetes API. This can be achieved using `kubectl` to proxy your requests to the API.

Discover all nodes in your cluster by running the following command:

```
kubectl get nodes
```

Next, initiate a proxy with kubectl on a local port of your choice. In this example we will use 8080:

```
kubectl proxy --port=8080
```

With this running, in a separate terminal run the following command for each node:

```
export NODE_NAME=my-node-name
curl http://localhost:8080/api/v1/nodes/${NODE_NAME}/proxy/configz
```

The curl command will return the API response which will be a JSON formatted string representing the Kubelet configuration.

Verify that a client certificate authority file is configured with `"authentication": { "x509": {"clientCAFile": <path/to/client-ca-file> } }"` in the API response.

Remediation:

Remediation Method 1:

If configuring via the Kubelet config file, you first need to locate the file.

To do this, SSH to each node and execute the following command to find the kubelet process:

```
ps -ef | grep kubelet
```

The output of the above command provides details of the active kubelet process, from which we can see the location of the configuration file provided to the kubelet service with the `--config` argument. The file can be viewed with a command such as `more` or `less`, like so:

```
sudo less /path/to/kubelet-config.json
```

Configure the client certificate authority file by setting the following parameter appropriately:

```
"authentication": { "x509": {"clientCAFile": <path/to/client-ca-file> } }"
```

Remediation Method 2:
If using executable arguments, edit the kubelet service file on each worker node and ensure the below parameters are part of the `KUBELET_ARGS` variable string.
For systems using `systemd`, such as the Amazon EKS Optimised Amazon Linux or Bottlerocket AMIs, then this file can be found at `/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf`. Otherwise, you may need to look up documentation for your chosen operating system to determine which service manager is configured:

```
--client-ca-file=<path/to/client-ca-file>
```

For Both Remediation Steps:
Based on your system, restart the `kubelet` service and check the service status.
The following example is for operating systems using `systemd`, such as the Amazon EKS Optimised Amazon Linux or Bottlerocket AMIs, and invokes the `systemctl` command. If `systemctl` is not available then you will need to look up documentation for your chosen operating system to determine which service manager is configured:

```
systemctl daemon-reload
systemctl restart kubelet.service
systemctl status kubelet -l
```

Default Value:

See the EKS documentation for the default value.

References:

1. https://kubernetes.io/docs/reference/command-line-tools-reference/kubelet/
2. https://kubernetes.io/docs/reference/access-authn-authz/kubelet-authn-authz/#kubelet-authentication
3. https://kubernetes.io/docs/reference/config-api/kubelet-config.v1beta1/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.10 Encrypt Sensitive Data in Transit Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).		●	●
v7	14.4 Encrypt All Sensitive Information in Transit Encrypt all sensitive information in transit.		●	●

3.2.4 Ensure that the --read-only-port is disabled (Manual)

Profile Applicability:

- Level 1

Description:

Disable the read-only port.

Rationale:

The Kubelet process provides a read-only API in addition to the main Kubelet API. Unauthenticated access is provided to this read-only API which could possibly retrieve potentially sensitive information about the cluster.

Impact:

Removal of the read-only port will require that any service which made use of it will need to be re-configured to use the main Kubelet API.

Audit:

If using a Kubelet configuration file, check that there is an entry for `authentication: anonymous: enabled` set to `0`.
First, SSH to the relevant node:
Run the following command on each node to find the appropriate Kubelet config file:

```
ps -ef | grep kubelet
```

The output of the above command should return something similar to `--config /etc/kubernetes/kubelet/kubelet-config.json` which is the location of the Kubelet config file.
Open the Kubelet config file:

```
cat /etc/kubernetes/kubelet/kubelet-config.json
```

Verify that the `--read-only-port` argument exists and is set to `0`.
If the `--read-only-port` argument is not present, check that there is a Kubelet config file specified by `--config`. Check that if there is a `readOnlyPort` entry in the file, it is set to `0`.

Remediation:

If modifying the Kubelet config file, edit the kubelet-config.json file `/etc/kubernetes/kubelet/kubelet-config.json` and set the below parameter to 0

```
"readOnlyPort": 0
```

If using executable arguments, edit the kubelet service file `/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf` on each worker node and add the below parameter at the end of the `KUBELET_ARGS` variable string.

```
--read-only-port=0
```

For each remediation:
Based on your system, restart the `kubelet` service and check status

```
systemctl daemon-reload
systemctl restart kubelet.service
systemctl status kubelet -l
```

Default Value:

See the Amazon EKS documentation for the default value.

References:

1. https://kubernetes.io/docs/admin/kubelet/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	12.6 Use of Secure Network Management and Communication Protocols Use secure network management and communication protocols (e.g., 802.1X, Wi-Fi Protected Access 2 (WPA2) Enterprise or greater).		●	●
v7	9.2 Ensure Only Approved Ports, Protocols and Services Are Running Ensure that only network ports, protocols, and services listening on a system with validated business needs, are running on each system.		●	●

3.2.5 Ensure that the --streaming-connection-idle-timeout argument is not set to 0 (Automated)

Profile Applicability:

- Level 1

Description:

Do not disable timeouts on streaming connections.

Rationale:

Setting idle timeouts ensures that you are protected against Denial-of-Service attacks, inactive connections and running out of ephemeral ports.

Note: By default, `--streaming-connection-idle-timeout` is set to 4 hours which might be too high for your environment. Setting this as appropriate would additionally ensure that such streaming connections are timed out after serving legitimate use cases.

Impact:

Long-lived connections could be interrupted.

Audit:

Audit Method 1:
First, SSH to the relevant node:
Run the following command on each node to find the running kubelet process:

```
ps -ef | grep kubelet
```

If the command line for the process includes the argument `streaming-connection-idle-timeout` verify that it is not set to 0.
If the `streaming-connection-idle-timeout` argument is not present in the output of the above command, refer instead to the `config` argument that specifies the location of the Kubelet config file e.g. `--config /etc/kubernetes/kubelet/kubelet-config.json`.
Open the Kubelet config file:

```
cat /etc/kubernetes/kubelet/kubelet-config.json
```

Verify that the `streamingConnectionIdleTimeout` argument is not set to `0`.
Audit Method 2:
If using the api configz endpoint consider searching for the status of `"streamingConnectionIdleTimeout":"4h0m0s"` by extracting the live configuration from the nodes running kubelet.
Set the local proxy port and the following variables and provide proxy port number and node name;
`HOSTNAME_PORT="localhost-and-port-number"`
`NODE_NAME="The-Name-Of-Node-To-Extract-Configuration" from the output of "kubectl get nodes"`

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 (example host and port number)
export NODE_NAME=ip-192.168.31.226.ec2.internal (example node name from
"kubectl get nodes")

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"
```

Remediation:

Remediation Method 1:
If modifying the Kubelet config file, edit the kubelet-config.json file `/etc/kubernetes/kubelet/kubelet-config.json` and set the below parameter to a non-zero value in the format of #h#m#s

```
"streamingConnectionIdleTimeout": "4h0m0s"
```

You should ensure that the kubelet service file `/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf` does not specify a `--streaming-connection-idle-timeout` argument because it would override the Kubelet config file.

Remediation Method 2:
If using executable arguments, edit the kubelet service file `/etc/systemd/system/kubelet.service.d/10-kubelet-args.conf` on each worker node and add the below parameter at the end of the `KUBELET_ARGS` variable string.

```
--streaming-connection-idle-timeout=4h0m0s
```

Remediation Method 3:
If using the api configz endpoint consider searching for the status of `"streamingConnectionIdleTimeout":` by extracting the live configuration from the nodes running kubelet.
**See detailed step-by-step configmap procedures in Reconfigure a Node's Kubelet in a Live Cluster, and then rerun the curl statement from audit process to check for kubelet configuration changes

```
kubectl proxy --port=8001 &

export HOSTNAME_PORT=localhost:8001 (example host and port number)
export NODE_NAME=ip-192.168.31.226.ec2.internal (example node name from
"kubectl get nodes")

curl -sSL "http://${HOSTNAME_PORT}/api/v1/nodes/${NODE_NAME}/proxy/configz"
```

For all three remediations:
Based on your system, restart the `kubelet` service and check status